	T.C (Average case)	S.C	Stability	In-place	Best case T.C.	Worst case
Selection sort	$O(n^2)$	$O(1)$	No	Yes	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n^2)$	$O(1)$	Yes	Yes	$O(n)$ ⟸	$O(n^2)$
Merge sort (merge)	$O(n\log n)$	$O(n)$	Yes	No	$O(n\log n)$	$O(n\log n)$
Quicksort (partition)	$O(n\log n)$	$O(1)$	No	Yes	$O(n\log n)$	$O(n^2)$ ⟸

↳ Week 3

$O(1)$ = Data storage

Insertion:

[5, 4, 6, 2, 8, 1]
[4, 5, 6, 2, 8, 1]
[1, 2, 3, 4, 5, 6]

1	1
2	1
⋮	1
n-1	1

$k = n-1$

Best case scenario: $O(1 \times n-1) = O(n-1) = O(n)$

sum of k natural nos $= \frac{k(k+1)}{2}$ $\quad \frac{(n-1)n}{2}$

Quicksort: Partition

[_]

```
01 def partition(L, lower, upper):
02     pivot = L[lower]
03     i = lower
04     for j in range(lower+1, upper+1):
05         if L[j] < pivot:
06             i += 1
07             L[i], L[j] = L[j], L[i]
08     L[i], L[lower] = L[lower], L[i]
09     return i
10 def quicksort(L, lower, upper)
11     if lower >= upper:
12         return
13     piv_id = partition(L, lower, upper)
14     quicksort(L, lower, piv_id-1)
15     quicksort(L, piv_id+1, upper)
```

Worst case example:

[1,2,3,4,5,6] n-1 comparisons → piv_id=0 } as already sorted
n-2 comparisons → piv_id=1 }

Hence, $(n-1)+(n-2)+\cdots+2+1 = \frac{n(n-1)}{2} = O(n^2)$

Worst case!! for Quick sort

[3,4,6,8,1,2,5]

[2,1,3,8,4,6,5]

piv_id=2

lower=0 upper=1 [1,2,3,8,4,6,5]

lower=3 upper=6 [1,2,3,5,4,6,8]

piv_id=6

lower=3 upper=5 [1,2,3,4,5,6,8]

lower=7 upper=6 return

piv_id=4

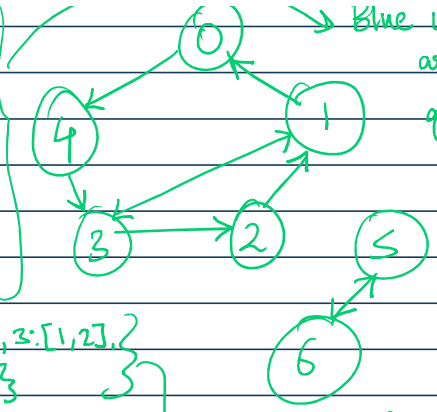
lower=3 upper=3 return

lower=5 upper=5 return

[[0,1,0,0,1,0,0],
[1,0,1,1,0,0,0],

Blue underlined 1s are assuming the

[[0,1,0,0,1,0,0],
[1,0,1,1,0,0,0],
[0,1,0,1,0,0,0],
[0,1,1,0,1,0,0],
[1,0,0,1,0,0,0],
[0,0,0,0,0,0,1],
[0,0,0,0,0,1,0]]



Blue underlined 1s are assuming the graph is undirected

return return

{0:[4],1:[0,3],2:[1],3:[1,2],
4:[3],5:[6],6:[5]}

For the given directed graph.

https://devamm007.github.io/pdsa-ta-t32026/22-10-2026

Old Session Notes
Extended thinking
& implementation

Also, Implement BFS using Adjacency Matrix

```
# Unweighted Graph: Adjacency List representation
adjl = {
    '0': ['1', '2'],
    '1': ['0', '3'],
    '2': ['0', '4'],
    '3': ['1'],
    '4': ['2']
}

# --- Approach 1: BFS using a standard Python list as queue ---
# Note: queue.pop(0) takes O(V) time to shift elements, making overall TC: O(V^2 + E)
def BFS_list(adjl, s):
  visited = set()
  queue = [s]
  while queue:
    node = queue.pop(0)
    print(node)
    visited.add(node)
    for neigh in adjl[node]:
      if neigh not in visited:
        queue.append(neigh)
  return visited
```

```
# --- Approach 2: BFS using collections.deque (Recommended & Optimal) ---
# TC: O(V + E) because deque.popleft() is O(1) amortized
# SC: O(V) for queue and visited set
def BFS(adjl, s):
  visited = set()
  queue = deque([s])
  while queue:
    node = queue.popleft()
    print(node)
    visited.add(node)
    for neigh in adjl[node]:
      if neigh not in visited:
        queue.append(neigh)
  return visited
```

Same algo, what's different?

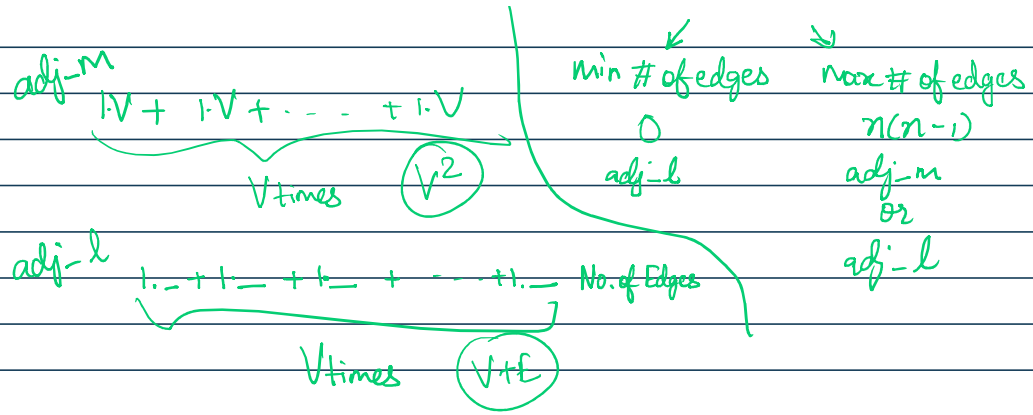
```
def BFS(am, s):
    visited = set()
    queue = deque()  # from collections import deque
    while queue:
        node = queue.popleft()
        print(node)
        visited.add(node)
        for neigh in range(len(am)):
            if am[node][neigh] == 1 and neigh not in visited:
                queue.append(neigh)
    return visited
```

Time complexity: $O(V^2)$

adj-M

n vertices

min # of edges max # of edges


adj_m
1·V + 1·V + · · · + 1·V
V times
V²
adj_l
1·_ + 1·_ + 1·_ + · · · + 1·_
No. of Edges
V times
V+E
min # of edges
0
adj_l
max # of edges
n(n-1)
adj_m
or
adj_l